```cpp
#pragma once
#include <iostream>
#include <vector>
#include <string>
#include <fstream>
#include <algorithm>
#include <cmath>
#include <random>

#include "matrix.hpp"
#include "capaEmbedding.hpp"
#include "PositionalEncoding.hpp"
#include "BloqueTransformer.hpp"
#include "capaDensa.hpp"
#include "FuncionesPerdida.hpp"

class CetzModelo {
private:
    size_t vocabSize_;
    size_t dimModel_;
    size_t maxSeqLen_;
    size_t numBloques_;

    capaEmbedding emb_;
    PositionalEncoding posEnc_;
    std::vector<BloqueTransformer> bloques_;
    capaDensa lmHead_;

    // Motor interno de muestreo estocástico
    size_t muestrearToken(const Matriz& probas, double temperatura, int topK, double topP
) {
        size_t n = probas.filas();
        std::vector<std::pair<double, size_t>> items;

        double suma = 0.0;
        std::vector<double> pAjustadas(n);
        for (size_t i = 0; i < n; ++i) {
            double p = std::pow(probas(i, 0), 1.0 / std::max(temperatura, 1e-5));
            pAjustadas[i] = p;
            suma += p;
        }

        for (size_t i = 0; i < n; ++i) {
            pAjustadas[i] /= suma;
            items.push_back({pAjustadas[i], i});
        }

        std::sort(items.rbegin(), items.rend());

        if (topK > 0 && static_cast<size_t>(topK) < items.size()) {
            items.resize(topK);
        }

        if (topP < 1.0) {
            double acum = 0.0;
            size_t corte = items.size();
            for (size_t i = 0; i < items.size(); ++i) {
                acum += items[i].first;
                if (acum >= topP) { corte = i + 1; break; }
            }
            items.resize(corte);
        }

        double sumaFiltrada = 0.0;
        for (const auto& item : items) sumaFiltrada += item.first;
```

```
        std::vector<double> probasFinales;
        std::vector<size_t> indicesFinales;
        for (const auto& item : items) {
            probasFinales.push_back(item.first / sumaFiltrada);
            indicesFinales.push_back(item.second);
        }

        static std::random_device rd;
        static std::mt19937 gen(rd());
        std::discrete_distribution<size_t> dist(probasFinales.begin(), probasFinales.end
());

        return indicesFinales[dist(gen)];
    }

public:
    CetzModelo(size_t vocabSize, size_t dimModel, size_t numHeads, size_t dimFFN, size_t
numBloques, size_t maxSeqLen)
        : vocabSize_(vocabSize),
          dimModel_(dimModel),
          maxSeqLen_(maxSeqLen),
          numBloques_(numBloques),
          emb_(vocabSize, dimModel),
          posEnc_(maxSeqLen, dimModel),
          lmHead_(dimModel, vocabSize, sigmoide, derivadaSigmoide)
    {
        lmHead_.usarSoftmax();
        for (size_t b = 0; b < numBloques_; ++b) {
            bloques_.emplace_back(dimModel, numHeads, dimFFN);
        }
    }
    ~CetzModelo(){
    }

    // Forward Pass completo a través de toda la profundidad de la red
    Matriz forward(const std::vector<size_t>& seqEntrada) {
        size_t seqLen = seqEntrada.size();
        Matriz X(dimModel_, seqLen);

        for (size_t pos = 0; pos < seqLen; ++pos) {
            Matriz vEmb = emb_.forward(seqEntrada[pos]);
            Matriz vPos = posEnc_.obtenerPosicion(pos);
            Matriz vFinal = vEmb + vPos;
            for (size_t d = 0; d < dimModel_; ++d) {
                X(d, pos) = vFinal(d, 0);
            }
        }

        Matriz act = X;
        for (size_t b = 0; b < numBloques_; ++b) {
            act = bloques_[b].forward(act);
        }

        return act;
    }

    // Paso de entrenamiento individual (Forward + Loss + Backward)
    void trainStep(const std::vector<size_t>& seqEntrada, const std::vector<size_t>&
seqObjetivo, double lr, int& t) {
        size_t seqLen = seqEntrada.size();

        std::vector<Matriz> objetivosOneHot;
        for (size_t targetId : seqObjetivo) {
            Matriz target(vocabSize_, 1, 0.0);
            target(targetId, 0) = 1.0;
            objetivosOneHot.push_back(target);
```

```
        }

        // 1. Forward Pass
        Matriz act = forward(seqEntrada);

        // 2. Backward Pass LM Head & Cross Entropy Loss
        Matriz gradActual(dimModel_, seqLen, 0.0);
        for (size_t pos = 0; pos < seqLen; ++pos) {
            Matriz tokenContextual(dimModel_, 1);
            for (size_t d = 0; d < dimModel_; ++d) {
                tokenContextual(d, 0) = act(d, pos);
            }

            Matriz pred = lmHead_.forward(tokenContextual);
            Matriz gradCross = gradienteCrossEntropy(pred, objetivosOneHot[pos]);

            Matriz gradBackToken = lmHead_.backward(gradCross, lr, t);
            for (size_t d = 0; d < dimModel_; ++d) {
                gradActual(d, pos) = gradBackToken(d, 0);
            }
        }

        // 3. Backward Pass a través de los Bloques Transformer
        for (int b = static_cast<int>(numBloques_) - 1; b >= 0; --b) {
            gradActual = bloques_[b].backward(gradActual, lr, t);
        }

        // 4. Backward Pass a través de Embeddings
        for (size_t pos = 0; pos < seqLen; ++pos) {
            Matriz gradEmb(dimModel_, 1);
            for (size_t d = 0; d < dimModel_; ++d) {
                gradEmb(d, 0) = gradActual(d, pos);
            }
            emb_.backward(gradEmb, lr, t);
        }

        t++;
    }

    // Inferencia y predicción del siguiente token
    // Inferencia con Penalización de Repetición (evita bucles infinitos)
    size_t predecirSiguiente(const std::vector<size_t>& contexto, double temperatura, int
topK, double topP, double repetitionPenalty = 1.3) {
        size_t seqLen = contexto.size();
        Matriz act = forward(contexto);

        Matriz ultimoVec(dimModel_, 1);
        for (size_t d = 0; d < dimModel_; ++d) {
            ultimoVec(d, 0) = act(d, seqLen - 1);
        }

        Matriz pred = lmHead_.forward(ultimoVec);

        // Aplicar penalización a los últimos 10 tokens del contexto
        size_t ventanaHistorial = std::min(seqLen, static_cast<size_t>(10));
        for (size_t i = seqLen - ventanaHistorial; i < seqLen; ++i) {
            size_t idRepetido = contexto[i];
            if (idRepetido < pred.filas()) {
                pred(idRepetido, 0) /= repetitionPenalty; // Reduce la probabilidad de
repetir
            }
        }

        return muestrearToken(pred, temperatura, topK, topP);
    }
```

```
    // Métodos de Persistencia
    bool guardar(const std::string& ruta) const {
        std::ofstream out(ruta, std::ios::binary);
        if (!out.is_open()) return false;
        emb_.guardarBinario(out);
        for (size_t b = 0; b < numBloques_; ++b) {
            bloques_[b].guardarBinario(out);
        }
        lmHead_.guardarBinario(out);
        out.close();
        return true;
    }

    bool cargar(const std::string& ruta) {
        std::ifstream in(ruta, std::ios::binary);
        if (!in.is_open()) return false;
        emb_.cargarBinario(in);
        for (size_t b = 0; b < numBloques_; ++b) {
            bloques_[b].cargarBinario(in);
        }
        lmHead_.cargarBinario(in);
        in.close();
        return true;
    }
};
```